UML Diagrams for C++ Programming

Illustrated study notes: how to draw diagrams, read every common symbol, determine arrow direction, and translate diagrams into C++ classes and behavior.

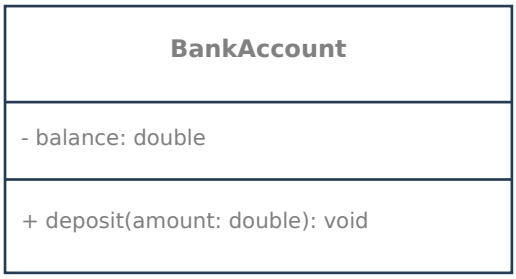
Focus: UML 2-style notation commonly examined in object-oriented C++ and Qt courses.

1. What UML is and how to choose a diagram

Unified Modeling Language (UML) is a visual notation for describing software structure and behavior. A UML diagram is not C++ source code: it communicates design decisions that can be implemented in C++.

Diagram	Main purpose
Class	Classes, fields, operations, inheritance and object relationships
Object	Specific instances and their values at a moment
Sequence	Messages exchanged over time
Activity	Workflow, decisions, parallel actions
State machine	States of one object and event-triggered transitions
Use case	Actors and user-visible system goals
Component / deployment	Modules, interfaces, processes and hardware nodes

2. Drawing a UML class



Three compartments: name / attributes / operations

Draw a rectangle with up to three compartments: class name at top, attributes in the middle, operations at bottom. A class may omit empty compartments. Italic class names or operations denote abstract elements; underlined attributes/operations denote static members.

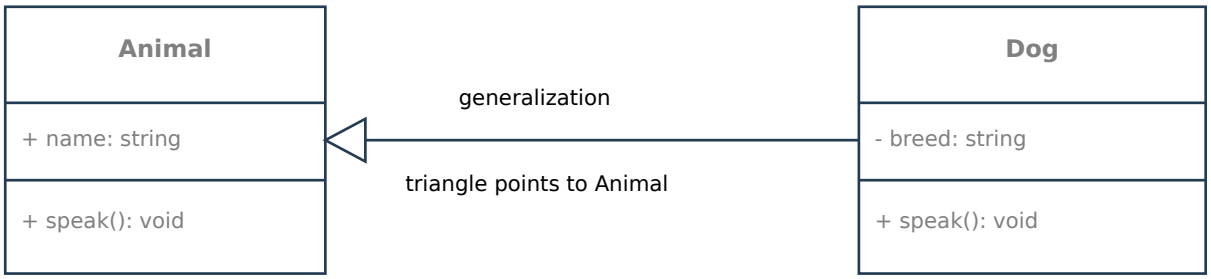
Notation	Meaning in C++
+	public
-	private

Notation	Meaning in C++
#	protected
~	package visibility (no exact standard C++ access equivalent)
name: Type	field or parameter and its type
method(x: int): bool	method name, typed argument, return type
= value	default value
/property	derived property (calculated)
{readOnly}	constraint: should not be modified
«interface»	interface-like abstraction (usually pure virtual C++ class)

```
class BankAccount {
private:
    double balance;
public:
    void deposit(double amount);
};
```

3. Relationships: lines, arrowheads and what they point toward

Golden rule: Identify the symbol at the END of the line. For inheritance, the hollow triangle points to the base/parent. For dependency, the dashed open arrow points to the thing being used. For aggregation/composition, the diamond sits at the whole/owner end, not at the part.

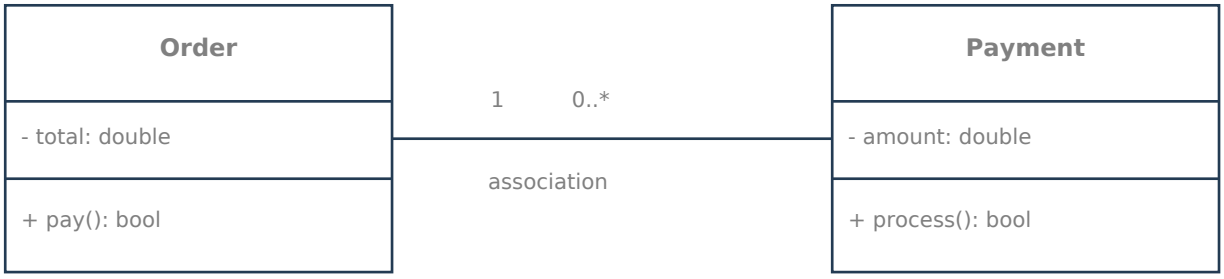


Relationship / line	End symbol and direction	C++ interpretation
Generalization (inheritance): solid line	Hollow triangle points to parent/base	class Dog : public Animal
Realization: dashed line	Hollow triangle points to interface/contract	class FileLogger : public ILogger

Relationship / line	End symbol and direction	C++ interpretation
Association: solid line	No arrow if navigability unspecified; open arrow if shown points to navigable target	Member pointer/reference or logical link
Directed association: solid line	Open arrow points to object reachable from source	Order can access Customer
Dependency: dashed line	Open arrow points to element depended on	Parameter, local variable, call, include/use
Aggregation: solid line	Hollow diamond at whole end	Weak whole-part; parts may exist independently
Composition: solid line	Filled diamond at whole end	Strong whole-part; part lifetime controlled by whole (conceptual)

Important: Association navigability is not always explicitly specified in UML. A line without an arrow does not automatically prove two-way navigation. Aggregation does not guarantee a particular C++ ownership mechanism; composition is a lifecycle design constraint, often implemented with value members or unique ownership.

4. Cardinality (multiplicity)



Label near endpoint	Meaning
1	Exactly one object at that end
0..1	Optional: zero or one
* or 0..*	Any number including zero
1..*	At least one
2..5	Between two and five inclusive
3	Exactly three

Read multiplicity from the opposite object: if Customer 1 ---- 0..* Order, each Customer may have many Orders; each Order relates to exactly one Customer. The label is placed beside the class whose number it describes.

5. C++ mapping examples

```
class Animal {

public:

    virtual ~Animal() = default;

    virtual void speak() const = 0;

};

class Dog : public Animal {

public:

    void speak() const override;

};
```

Inheritance: Dog is an Animal. The hollow triangle points from Dog toward Animal. Realization looks similar but uses a dashed line to the interface.

```
class Engine {};

class Car {

    Engine engine; // composition: engine is a value member

};

class Team {

    std::vector<Player*> players; // possible aggregation

};
```

Composition is about ownership/lifecycle, not just syntax. A raw pointer alone does not establish aggregation. Modern C++ often uses std::unique_ptr for exclusive ownership and std::shared_ptr only for genuinely shared ownership.

```
class Invoice {

public:

    void print(Printer& printer); // dependency

};

class Order {

    Customer* customer = nullptr; // navigable association

};
```

6. Step-by-step: draw a class diagram from C++

1. List classes and interfaces. 2. Draw one class rectangle for each. 3. Add important fields and methods with +, -, #. 4. Connect derived class to base with a solid line and hollow triangle at base. 5. Connect usage-only relationships with dashed open-arrow dependencies. 6. Add associations for persistent links. 7. Add diamonds only when whole-part semantics are justified. 8. Put multiplicities at both ends where known. 9. Verify arrowheads and ownership against the code and requirements.

7. Sequence diagrams: messages over time

Participants are rectangles across the top; dashed vertical lifelines run downward. Time proceeds from top to bottom. Horizontal arrows represent messages: the arrowhead points to the receiver. Narrow activation bars show execution intervals.

Symbol	Interpretation
Solid horizontal line + filled arrowhead	Synchronous call: caller typically waits
Solid horizontal line + open arrowhead	Asynchronous message: sender need not wait
Dashed line + open arrowhead	Reply/return (often omitted)
X at end of lifeline	Object destruction
«create» message	Object creation; lifeline begins at creation
alt / opt / loop frame	Alternatives / optional branch / repetition

Example: User -> Controller: checkout(); Controller -> PaymentService: charge(); PaymentService --> Controller: approved. Calls point to the callee; replies point back to the original caller.

8. Activity diagrams: control flow

Shape / mark	Meaning
Filled black circle	Initial node: workflow starts
Rounded rectangle	Action/activity step
Diamond	Decision or merge; guards such as [valid] / [invalid] label outgoing paths
Thick bar	Fork (one flow splits into concurrent flows) or join (wait for flows)
Bullseye circle	Activity final: entire activity ends
Circle with X	Flow final: ends only that flow
Arrow	Control or object flow direction

Example: Start -> Enter PIN -> decision [correct?] -> [yes] Dispense cash -> End; [no] Display error -> retry. A decision diamond selects a guarded branch; a merge diamond recombines alternatives.

9. State machine diagrams

States are rounded rectangles; transitions are directed arrows from source state to destination state. Label a transition as event [guard] / action. A filled circle is the initial pseudostate; a bullseye is a final state. A self-loop returns to the same state.

Example: Idle -- insertCard --> CardInserted -- [PIN valid] / authorize --> Authorized -- withdraw --> Idle. Guards are Boolean conditions; actions occur when the transition fires. State diagrams model the lifecycle of one object, unlike activity diagrams which model workflow.

10. Use case diagrams

Stick figures are actors outside the system boundary rectangle. Ellipses are use cases inside. Solid lines associate actors with use cases (normally no directional meaning). «include» is a dashed arrow from the including use case toward the included use case. «extend» is a dashed arrow from the extension use case toward the base use case. Generalization uses a hollow triangle pointing toward the more general actor/use case.

11. Component and deployment diagrams

Component diagrams show software components and provided/required interfaces. A lollipop (small circle) represents a provided interface; a socket (semicircle) represents a required interface. Dependencies point toward the required supplier. Deployment diagrams show nodes (3D-style boxes), execution environments and deployed artifacts; communication paths connect nodes.

12. Common exam mistakes and quick memory aids

Mistake	Correct reading
Triangle pointing at derived class	Incorrect: generalization triangle points at base class
Diamond pointing at part	Incorrect: diamond is attached to whole/owner
Dashed arrow treated as inheritance	Dashed open arrow is dependency; dashed hollow triangle is realization
Multiplicity read beside wrong class	A number describes instances at the endpoint beside it
Filled arrow in sequence read as return	Solid filled arrow usually indicates synchronous call
All diamonds interpreted as C++ pointers	Ownership is a semantic rule; implementation varies
Every solid line assumed bidirectional	Unmarked navigability may be unspecified

13. Mini worked design: online store

Requirements: One Customer places zero or more Orders; each Order has one or more OrderLines; each OrderLine refers to one Product; CardPayment implements PaymentMethod.

Draw Customer -- Order as an association with 1 near Customer and 0..* near Order. Draw a filled diamond at Order on the Order--OrderLine connection, with 1 near Order and 1..* near OrderLine. Draw OrderLine -- Product with 1 near Product. Draw CardPayment - - - hollow triangle -> PaymentMethod.

```
struct Product { std::string title; };

struct OrderLine { Product* product; int quantity; };

struct Order {

    std::vector<OrderLine> lines;

    Customer* customer;

};
```

```
struct PaymentMethod {

    virtual ~PaymentMethod() = default;

    virtual bool pay(double) = 0;

};

struct CardPayment : PaymentMethod {

    bool pay(double) override;

};
```

Revision check: Identify the parent, whole, part, receiver, dependent, and multiplicity endpoint before interpreting any arrow. This eliminates most direction-related UML errors.